Lecture Notes: TwinCAT 3

Basics and Installation

March 17, 2026

Contents

1 The Evolution of PLCs **2**
- 1.1 Pre-PLC Era . . . 2
- 1.2 The First PLCs . . . 2
- 1.3 PC-Based Control . . . 2

2 TwinCAT 3 Architecture **2**
- 2.1 XAE (eXtended Automation Engineering) . . . 2
- 2.2 XAR (eXtended Automation Runtime) . . . 2

3 Installation and Development Environment **3**
- 3.1 TcXaeShell . . . 3
- 3.2 Visual Studio Integration . . . 3
- 3.3 Licensing . . . 3
- 3.4 Local Execution . . . 3

4 System States and Operating Modes **5**
- 4.1 Blue (CONFIG Mode) . . . 5
- 4.2 Green (RUN Mode) . . . 5
- 4.3 Yellow (STOP/EXCEPTION Mode) . . . 5

5 Connectivity and Actuation **7**
- 5.1 Sensors . . . 7
- 5.2 Actuators . . . 7
- 5.3 Communication . . . 7

Quick Reference Card **8**

1 The Evolution of PLCs

1.1 Pre-PLC Era

Industrial processes were automated using **relay-based control systems**. These systems required significant space and cabling, were difficult to maintain, and a single wire breakage could disable the entire system.

1.2 The First PLCs

In the late 1960s, General Motors and Allen-Bradley developed electronic replacements for relays. The **Modicon 084** (1969) was the first PLC, though the **Modicon 184** (1973) became the first mass-market success.

1.3 PC-Based Control

Founded in 1980, Beckhoff introduced the shift from proprietary hardware to **standard PC-based control**. **TwinCAT** (The Windows Control and Automation Technology) was released in 1996, allowing standard Windows PCs to function as real-time controllers.

Year	Milestone	Company	Significance
1969	Modicon 084	Modicon	First PLC
1973	Modicon 184	Modicon	First commercial success
1980	Company Founded	Beckhoff	PC-based control pioneer
1996	TwinCAT 1	Beckhoff	Windows-based PLC runtime
2010	TwinCAT 3	Beckhoff	Visual Studio integration

Table 1: Key Milestones in PLC History

Note: TwinCAT stands for "The Windows Control and Automation Technology." Unlike traditional PLCs with proprietary hardware, TwinCAT transforms any standard Windows PC into a real-time controller, significantly reducing hardware costs and increasing flexibility.

2 TwinCAT 3 Architecture

TwinCAT 3 consists of two primary components that communicate via the **ADS protocol**:

2.1 XAE (eXtended Automation Engineering)

- **Role:** The development environment used to create, edit, and compile projects.
- **Platform:** Must be installed on a **Windows machine**.
- **IDE:** Based on **Microsoft Visual Studio**. It includes the **CODESYS compiler**, which is a standard in the automation industry.

2.2 XAR (eXtended Automation Runtime)

- **Role:** The software that actually executes the code in **real-time**.

- **Components:** Includes the real-time kernel for deterministic behavior and drivers for fieldbus protocols like **EtherCAT**.
- **Platform:** Can run on Windows or Beckhoff's **Tc/BSD** (FreeBSD-based) operating system.

XAE (Engineering)	ADS Protocol	**XAR** (Runtime)
Visual Studio IDE CODESYS Compiler Project Configuration		Real-time Kernel EtherCAT Drivers I/O Processing
Windows PC / Beckhoff IPC		

Figure 1: TwinCAT 3 Architecture Overview

Tip: The XAE and XAR can run on the same machine (common during development) or on separate machines (typical in production). The ADS protocol handles all communication between them, regardless of whether they are local or networked.

3 Installation and Development Environment

3.1 TcXaeShell

If you do not have Visual Studio installed, the TwinCAT installer bundles **TcXaeShell**, a version of Visual Studio dedicated to PLC programming.

3.2 Visual Studio Integration

If you already have Visual Studio (Community, Professional, or Enterprise), TwinCAT can integrate directly into those versions. This is necessary if you plan to program in other languages like **C# or C++** alongside your PLC code.

3.3 Licensing

TwinCAT 3 is **free for development and testing**. You can generate **7-day trial licenses** indefinitely by entering a CAPTCHA.

3.4 Local Execution

Because the XAE installation includes a local XAR, you can test and run your software directly on your development PC or a virtual machine without needing physical PLC hardware.

Installation Option	**Use Case**	**Requirements**
TcXaeShell Only	PLC development only	None
VS Integration	Mixed PLC + IT development	Visual Studio installed
XAR Only	Production runtime	Licensed for deployment

Table 2: TwinCAT Installation Options

Example: Basic Variable Declaration

```
// PROGRAM: MAIN
// Basic variable declaration example for TwinCAT 3
// This code is created within the XAE IDE (TcXaeShell or Visual Studio)

PROGRAM MAIN
VAR
    // ====================================================
    // BOOLEAN Variables
    // Used for digital I/O, flags, and logical operations
    // Can only be TRUE or FALSE
    // ====================================================
    bStartButton     : BOOL;              // Input: Start pushbutton
    bStopButton      : BOOL;              // Input: Stop pushbutton
    bMotorRunning    : BOOL := FALSE;     // Status flag with initial value
    bAlarmActive     : BOOL := FALSE;     // Alarm indicator
    bSafetyOK        : BOOL := TRUE;      // Safety circuit status

    // ====================================================
    // INTEGER Variables
    // Used for counters, setpoints, and whole numbers
    // INT is 16-bit signed: -32768 to 32767
    // ====================================================
    nCounter         : INT := 0;          // Part counter
    nSetpoint        : INT := 100;        // Speed setpoint
    nErrorCode       : INT;               // Error code storage
    nBatchSize       : INT := 50;         // Batch quantity

    // ====================================================
    // Additional common types for reference
    // ====================================================
    fTemperature     : REAL := 20.0;      // 32-bit floating point
    tCycleTime       : TIME := T#100MS;   // Time duration
    sProductName     : STRING := 'Widget';// Text string

END_VAR

// Program logic would follow here...
```

```
// Extended example showing variable organization

PROGRAM MAIN
VAR
    // Group related variables with comments for clarity

    // === INPUTS (from field devices) ===
    bSensor1         : BOOL;      // Proximity sensor 1
    bSensor2         : BOOL;      // Proximity sensor 2
    nAnalogInput     : INT;       // 0-10V analog input (0-32767)

    // === OUTPUTS (to field devices) ===
    bValve1          : BOOL;      // Solenoid valve 1
    bValve2          : BOOL;      // Solenoid valve 2
    nAnalogOutput    : INT;       // 0-10V analog output

```

```
    // === INTERNAL VARIABLES ===
    nState          : INT := 0; // State machine variable
    bFirstScan      : BOOL := TRUE;  // First scan flag

END_VAR

// Variables are now accessible in the program code
```

Note: Variable declarations in Structured Text follow the pattern: `VariableName : DataType := InitialValue;`. The initial value is optional—if omitted, variables default to zero (for numbers) or FALSE (for BOOL).

4 System States and Operating Modes

The state of the TwinCAT runtime is indicated by the color of the TwinCAT icon in the Windows taskbar:

4.1 Blue (CONFIG Mode)

Used when making changes to the system configuration, adding hardware, or upgrading software.

4.2 Green (RUN Mode)

The active state where TwinCAT executes all PLC tasks and programs.

4.3 Yellow (STOP/EXCEPTION Mode)

Indicates the runtime has stopped, often due to a software error like a **null pointer exception** or **stack overflow**.

Icon Color	Mode	State	Typical Action
Blue	CONFIG	Configuring	Add hardware, modify settings
Green	RUN	Executing	Normal operation
Yellow	STOP	Stopped	Debug, check for errors
Red	ERROR	Exception	Investigate crash cause

Table 3: TwinCAT Runtime States

Example: Simple Logic Assignment

```
// Simple logic assignment examples
// This code executes cyclically when TwinCAT is in RUN mode (green
    icon)

PROGRAM MAIN
VAR
    // Input variables (typically linked to physical inputs)
    bInput1         : BOOL;     // Digital input 1
    bInput2         : BOOL;     // Digital input 2
    bInput3         : BOOL;     // Digital input 3
    bEnableSwitch   : BOOL;     // Enable/disable switch
```

```

    // Output variables (typically linked to physical outputs)
    bOutput1         : BOOL;      // Digital output 1
    bOutput2         : BOOL;      // Digital output 2
    bAlarmLight      : BOOL;      // Alarm indicator lamp

    // Internal variables
    bConditionMet    : BOOL;      // Internal logic flag
END_VAR

// ====================================================
// BASIC AND LOGIC
// Output is TRUE only when BOTH inputs are TRUE
// Similar to series-connected relay contacts
// ====================================================
bOutput1 := bInput1 AND bInput2;

// ====================================================
// BASIC OR LOGIC
// Output is TRUE when EITHER input is TRUE
// Similar to parallel-connected relay contacts
// ====================================================
bOutput2 := bInput1 OR bInput2;

// ====================================================
// COMBINED LOGIC WITH ENABLE
// Output requires enable switch AND both inputs
// ====================================================
bConditionMet := bEnableSwitch AND (bInput1 AND bInput2);

// ====================================================
// NOT LOGIC (INVERSION)
// Alarm is active when enable is ON but condition NOT met
// ====================================================
bAlarmLight := bEnableSwitch AND NOT bConditionMet;

// ====================================================
// PRACTICAL EXAMPLE: Motor Start Circuit
// Motor runs when: Start pressed AND Stop not pressed AND Safety OK
// ====================================================
// bMotorRunning := bStartButton AND (NOT bStopButton) AND bSafetyOK;
```

```
// More complex logic examples

VAR
    bPumpEnable      : BOOL;
    bLevelHigh       : BOOL;
    bLevelLow        : BOOL;
    bPressureOK      : BOOL;

    bRunPump         : BOOL;
    bDrainValve      : BOOL;
    nOperatingMode   : INT;
END_VAR

// XOR logic - exclusive OR (one or the other, but not both)
bDrainValve := bLevelHigh XOR bLevelLow;

```

```
// Parentheses for complex expressions
// Pump runs if: enabled AND (level is low OR pressure is not OK)
bRunPump := bPumpEnable AND (bLevelLow OR NOT bPressureOK);

// Assignment with arithmetic
nOperatingMode := 1 + 2;  // nOperatingMode = 3

// Combined comparison
// bCondition := (nCounter > 10) AND (nCounter < 100);
```

Warning: When the TwinCAT icon turns yellow or red, the runtime has stopped executing. Check the error log in the XAE for details. Common causes include division by zero, null pointer access, or stack overflow from infinite recursion.

5 Connectivity and Actuation

Through fieldbus interfaces, TwinCAT communicates with various hardware components:

5.1 Sensors

Reads data from temperature, pressure, and position sensors.

5.2 Actuators

Controls devices such as **relays, valves, and electric motors**.

5.3 Communication

Users can read or write variables between the XAE and XAR (e.g., for HMI displays or data logging) using the ADS interface.

Category	Device Type	Signal Type	Data Type
Sensor	Proximity switch	Digital Input	BOOL
Sensor	Temperature probe	Analog Input	INT/REAL
Sensor	Encoder	Counter Input	DINT
Actuator	Solenoid valve	Digital Output	BOOL
Actuator	VFD (Motor drive)	Analog Output	INT
Actuator	Servo motor	Motion axis	Complex

Table 4: Common I/O Device Types

Field Devices	EtherCAT	**TwinCAT XAR**
Sensors		PLC Program
Actuators		Motion Control
Drives		Safety Logic

Figure 2: TwinCAT Connectivity Architecture

Tip: EtherCAT is Beckhoff's high-performance fieldbus protocol. It provides deterministic communication with cycle times as low as 100 microseconds, making it ideal for motion control and high-speed applications.

Quick Reference Card

TwinCAT Components

Component	Full Name	Purpose
XAE	eXtended Automation Engineering	Development environment
XAR	eXtended Automation Runtime	Real-time execution
ADS	Automation Device Specification	Communication protocol

Runtime States

Color	Mode	Description
Blue	CONFIG	Configuration/editing
Green	RUN	Normal operation
Yellow	STOP	Stopped (check errors)

Basic Data Types

Type	Size	Range	Example
BOOL	1 bit	TRUE/FALSE	`bFlag : BOOL := TRUE;`
INT	16 bits	-32768 to 32767	`nCount : INT := 0;`
DINT	32 bits	±2.1 billion	`nLarge : DINT;`
REAL	32 bits	Floating point	`fTemp : REAL := 25.5;`

Logic Operators

```
// AND - both must be TRUE
bResult := bA AND bB;

// OR - either can be TRUE
bResult := bA OR bB;

// NOT - inverts the value
bResult := NOT bA;

// XOR - exclusive OR
bResult := bA XOR bB;
```

Variable Declaration Syntax

```
VAR
    VariableName : DataType;                // No initial value
    VariableName : DataType := Value;       // With initial value
END_VAR
```

Last Updated: March 17, 2026